

个人项目使用大模型后人工修改代码一览表

所在源代码文件名	人工修改代码（标红）	备注
PrimaryGenerator.java	<pre>// 随机给表达式加括号（旧） if (operands >= 3 && random.nextBoolean()) { int b = 2 + random.nextInt(operands - 1); int leftIndex = 0; int rightIndex = b * 2; tokens.add(leftIndex, "("); tokens.add(rightIndex + 2, ")"); }</pre> <pre>// 简单随机加括号（可选） if (operands >= 3 && random.nextBoolean()) { // 选择从第0个数到第b个数加括号，b取值范围：1..(operands-1) // 这样可确保至少包含两个操作数，且不越界 int b = 1 + random.nextInt(operands - 1); // 在tokens层面插入括号 // 将第a个数到第b个数之间加上括号，简化处理：在对应位置插入 int leftIndex = 0; // 第a个数的索引（数在偶数位） int rightIndex = b * 2; // 第b个数的索引 tokens.add(leftIndex, "("); tokens.add(rightIndex + 2, ")"); }</pre>	原因说明 生成小学题目时会把所有“数”和“运算符”放到 tokens 列表中，结构是：数 运算符 数 运算符 数 ... ，长度是 $2 * operands - 12 * operands - 1$。 随机加括号时，先在索引 0 位置插入左括号 "(", 此举会让列表长度+1，且所有原有索引整体右移 1。 代码中原来这么取右括号位置： • $b = 2 + \text{random.nextInt}(\text{operands} - 1)$，所以 b 可能等于 operands。 • $\text{rightIndex} = b * 2$；如果 $b == \text{operands}$，则 $\text{rightIndex} = 2 * \text{operands}$。 • 之后又执行 <code>tokens.add(rightIndex + 2, ")")</code>。此时列表长度是 $2 * \text{operands}$，而要插入的位置是 $2 * \text{operands} + 2$，超过当前列表最大可插入位置（允许等于 size 作为尾部追加），因此抛出 <code>IndexOutOfBoundsException</code>。 简言之：右括号的索引计算越界，尤其当 b 被随机到最大值时。 变化点说明 将 b 的范围从可能取到 operands，改为只能取到 $\text{operands} - 1$

		(1..operands-1) ， 保证 rightIndex + 2 最大为 2 * operands，恰好等于当前列表长度，属于合法的“在尾部追加”的位置，不会越界。
	<pre>// 生成一个仅含 + - × ÷ 和括号 的表达式 private String makeExpression() { int operands = 1 + random.nextInt(5); // 操作数数量: 1-5 // ... 其余代码 }</pre>	当前代码中 operands = 1 + random.nextInt(5) 可能生成只有 1 个操作数的表达式，但小学题目应该至少有 2 个操作数才能形成有效的数学运算。 修改方案： 将操作数范围从 1-5 改为 2-5 即： operands = 2 + random.nextInt(4)，确保最少 2 个操作数 修改效果 修改前：可能生成如 "5" 这样的单操作数表达式（没有运算符） 修改后：至少生成如 "5 + 3" 这样的双操作数表达式，确保有实际的数学运算
	<pre>import java.util.*; 以下为修改后代码： import java.util.ArrayList; import java.util.List; import java.util.Random;</pre>	Google Java 编程规范(中文版) import 不要使用通配符，即： 不要出现类似这样的 import 语句： import java.util.*;
	<pre>以下为修改后代码（新增代码）： private String makeExpression() { int operands = 2 + random.nextInt(4); // 操作数数量: 2-5（确保至少2个操作数） List<Integer> nums = generateOperands(operands); List<String> ops = generateOperators(operands - 1); // 确保减法不产生负数 adjustForNonNegativeSubtraction(nums, ops); // 构建表达式 List<String> tokens = buildExpressionTokens(nums, ops); // 简单随机加括号（可选） if (operands >= 3 && random.nextBoolean()) { addParentheses(tokens, operands); } }</pre>	小学题目会出现负数，新增函数让表达式的结果不为负，符合小学生能力水平。 新增了防止负数的逻辑： - adjustForNonNegativeSubtraction() 方法专门处理减法运算 - 当遇到减法运算符时，检查被减数是否小于减数 - 如果被减数小于减数，则交换两个数的位置，确保结果非负

	<pre> } return String.join(" ", tokens); }</pre> <pre>private String makeExpression() { int operands = 2 + random.nextInt(4); // 操作数数量: 2-5 (确保至少 2 个操作数) List<Integer> nums = generateOperands(operands); List<String> ops = generateOperators(operands - 1); // 确保减法不产生负数 adjustForNonNegativeSubtraction(nums, ops); // 构建表达式 List<String> tokens = buildExpressionTokens(nums, ops); // 简单随机加括号 (可选) if (operands >= 3 && random.nextBoolean()) { addParentheses(tokens, operands); } return String.join(" ", tokens); }</pre> <pre>// 调整操作数确保减法不产生负数 private void adjustForNonNegativeSubtraction(List<Integer> nums, List<String> ops) { for (int i = 0; i < ops.size(); i++) { if ("-".equals(ops.get(i))) { // 确保被减数大于等于减数 int minuend = nums.get(i); int subtrahend = nums.get(i + 1); if (minuend < subtrahend) { // 交换两个数, 确保被减数更大 nums.set(i, subtrahend); nums.set(i + 1, minuend); } } } }</pre>	
Auth Servi	<pre>import java.util.*;</pre> 以下为修改后代码:	Google Java 编程规范(中文版) import 不要使用通配符, 即:

ce.java	<pre>import java.util.HashMap; import java.util.Map; import java.util.Objects; import java.util.Optional;</pre>	不要出现类似这样的 import 语句：<code>import java.util.*;</code>
	以下为修改后代码（方法重构，划分模块）： <pre>// 初始化所有预设账号 private void initializeAccounts() { addPrimaryAccounts(); addMiddleAccounts(); addHighAccounts(); } // 添加小学账号 private void addPrimaryAccounts() { String[] usernames = {"张三 1", "张三 2", "张三 3"}; for (String username : usernames) { userToPassword.put(username, "123"); userToType.put(username, AccountType.PRIMARY); } } // 添加初中账号 private void addMiddleAccounts() { String[] usernames = {"李四 1", "李四 2", "李四 3"}; for (String username : usernames) { userToPassword.put(username, "123"); userToType.put(username, AccountType.MIDDLE); } } // 添加高中账号 private void addHighAccounts() { String[] usernames = {"王五 1", "王五 2", "王五 3"}; for (String username : usernames) { userToPassword.put(username, "123"); userToType.put(username, AccountType.HIGH); } }</pre>	附表 3 功能：方法代码行数<40 行 将长构造函数拆分为： • initializeAccounts() - 初始化所有账号（4 行） • addPrimaryAccounts() - 添加小学账号（8 行） • addMiddleAccounts() - 添加初中账号（8 行） • addHighAccounts() - 添加高中账号（8 行） 代码更加模块化和可读 每个方法职责单一，便于维护
Deduplicator.java	<pre>import java.util.*; import java.io.*; import java.nio.file.*;</pre>	Google Java 编程规范(中文版) import 不要使用通配符，即：不要出现类似这样的 import 语句：<code>import java.util.*;</code>

	<pre>import java.util.ArrayList; import java.util.HashSet; import java.util.LinkedHashSet; import java.util.List; import java.util.Set;</pre>	
High Gen erat or.java va	<pre>import java.util.*;</pre> 以下为修改后代码: <pre>import java.util.ArrayList; import java.util.List; import java.util.Random;</pre>	Google Java 编程规范(中文版) import 不要使用通配符, 即: 不要出现类似这样的 import 语句: import java.util.*;
Main .java	<pre>import java.util.*;</pre> 以下为修改后代码: <pre>import java.util.List; import java.util.Optional; import java.util.OptionalInt; import java.util.Scanner;</pre>	Google Java 编程规范(中文版) import 不要使用通配符, 即: 不要出现类似这样的 import 语句: import java.util.*;
	以下为修改后代码(方法重构, 划分模块): <pre>// 处理用户登录, 返回用户会话, 失败返回 null private static UserSession loginUser(ConsolePrompter prompter, AuthService authService) { while (true) { String line = prompter.readLine("请输入用户名和密码(用空格分隔): "); String[] parts = line.trim().split("\\s+", 2); if (parts.length != 2) { prompter.println("请输入正确的用户名、密码"); continue; } String username = parts[0]; String password = parts[1]; Optional<AccountType> typeOpt = authService.verify(username, password); if (!typeOpt.isPresent()) { prompter.println("请输入正确的用户名、密码"); continue; } AccountType currentType = typeOpt.get(); prompter.println("当前选择为 " + currentType.displayName() + " 出题"); return new UserSession(username, currentType); } }</pre>	附表 3 功能: 方法代码行数<40 行 • 将原来的长 main 方法拆分为: • loginUser() - 处理用户登录 (18 行) • handleUserSession() - 处理用户会话 (31 行) • generateAndSaveQuestions() - 生成并保存题目 (8 行) • 新增 UserSession 类来管理用户状态 代码更加模块化和可读 每个方法职责单一, 便于维护

```

// 处理登录后的用户会话
private static void handleUserSession(ConsolePrompter prompter,
UserSession session) {
    while (true) {
        String prompt = "准备生成 " +
session.getAccountType().displayName() + " 数学题目, 请输入生成题目数量(输入-1将退出当前用户, 重新登录): ";

        String input = prompter.readLine(prompt).trim();

        // 处理切换命令
        Optional<AccountType> switchType =
Validators.parseSwitchCommand(input);
        if (switchType.isPresent()) {
            session.setAccountType(switchType.get());
            prompter.println("准备生成 " +
session.getAccountType().displayName() + " 数学题目, 请输入生成题目数量");
            continue;
        } else if (Validators.isSwitchCommandLikeButInvalid(input))
{
            prompter.println("请输入小学、初中和高中三个选项中的一个");
        }

        continue;
    }

    // 处理数量输入
    OptionalInt countOpt = Validators.parseCount(input);
    if (!countOpt.isPresent()) {
        prompter.println("请输入 10-30 之间的数字, 或-1 退出登录");
        continue;
    }
    int count = countOpt.getAsInt();
    if (count == -1) {
        break; // 退出当前用户
    }

    // 生成并保存题目
    generateAndSaveQuestions(prompter, session, count);
}
}

```

```

// 生成并保存题目
private static void generateAndSaveQuestions(ConsolePrompter

```

```

prompter, UserSession session, int count) {
    ProblemGenerator generator =
GeneratorFactory.getGenerator(session.getAccountType());
    List<String> raw = generator.generate(count);
    Deduplicator deduplicator = new
Deduplicator(session.getUsername());
    List<String> unique = deduplicator.ensureUnique(raw, count,
generator);
    PaperSaver saver = new PaperSaver(session.getUsername());
    String filePath = saver.save(unique);
    prompter.println("已生成并保存: " + filePath);
}
}

```

```

// 用户会话类，保存用户名和账户类型
class UserSession {
    private final String username;
    private AccountType accountType;

```

```

    public UserSession(String username, AccountType accountType) {
        this.username = username;
        this.accountType = accountType;
    }

```

```

    public String getUsername() {
        return username;
    }

```

```

    public AccountType getAccountType() {
        return accountType;
    }

```

```

    public void setAccountType(AccountType accountType) {
        this.accountType = accountType;
    }
}

```

Midd `import java.util.*;`

Google Java 编程规范(中文版)

leGenerator.java	以下为修改后代码: <pre>import java.util.ArrayList; import java.util.List; import java.util.Random;</pre>	import 不要使用通配符，即：不要出现类似这样的 import 语句： import java.util.*;
PaperSaver.java	<pre>import java.util.*;</pre> 以下为修改后代码: <pre>import java.io.*; import java.nio.file.*; import java.time.*; import java.time.format.DateTimeFormatter; import java.util.List;</pre>	Google Java 编程规范(中文版) import 不要使用通配符，即：不要出现类似这样的 import 语句： import java.util.*;
ProblemGenerator.java	<pre>import java.util.*;</pre> 以下为修改后代码: <pre>import java.util.List;</pre>	Google Java 编程规范(中文版) import 不要使用通配符，即：不要出现类似这样的 import 语句： import java.util.*;
Validators.java	<pre>import java.util.*;</pre> 以下为修改后代码: <pre>import java.util.Optional; import java.util.OptionalInt;</pre>	Google Java 编程规范(中文版) import 不要使用通配符，即：不要出现类似这样的 import 语句： import java.util.*;

注：新增的语句，以及大模型生成的语句作了任何修改，都算人工修改代码。但是，表里面不要出现完全
是大模型生成后原封不动的代码。